

Calcul Parallèle

TP : Partie 1

Thomas STAVIS
(thomas.stavis@ens-lyon.fr)

Jonathan Rouzaud-Cornabas
(jonathan.rouzaud-cornabas@inria.fr)

Ce TP traite de manière théorique du parallélisme appliqué à certains problèmes. Il est un moyen de comprendre l'utilité du parallélisme, de savoir comment le mettre en œuvre, ainsi que d'introduire quelques-unes de ses subtilités et de ses limites. Ce TP comporte 2 parties : une sur les tris, et une autre sur la multiplication de matrices.

Rendu Un rapport devra être rendu pour ce TP. Il est conseillé de commencer à l'écrire pendant les séances puis de le reprendre chez vous avant de le rendre. Comme le TP est en 2 parties, vous devrez rendre 2 rapports. Vous devrez envoyer ces rapports dans un même mail à l'adresse "thomas.stavis@ens-lyon.fr", avec comme objet "[HPC GPU INSA] : <prénom> <nom>". **La date limite de réception de ce rapport est le 12/10/2026 à 18h.** Bien que vous puissiez réfléchir à ce TP en groupe pendant les séances et en dehors, **les rendus sont individuels.**

Notation La partie 1 de ce TP est sur 28 points. La note de cette partie sera de $\min(20, \text{votre note à la partie})/20$. La note finale du TP1 sera une moyenne des notes des 2 parties.

LLM L'utilisation de LLM pour répondre aux questions n'est pas pénalisée. Cependant, le TP n'est pas long et son but est que vous intégriez certains concepts. Si vous vous en aidez, merci de le noter dans les rapports en précisant comment vous les avez utilisés.

0 Rappel : Complexité en temps

La **complexité en temps** est une mesure théorique du temps nécessaire à l'exécution d'un algorithme. En fonction de la taille des paramètres donnés en entrée (taille de liste, matrice, etc.), elle décompte le nombre d'affectations, de comparaisons et d'opérations arithmétiques réalisées au cours de cet algorithme. Alors, en supposant que chacune de ces opérations prend un temps constant C secondes à être exécuté, nous pouvons en déduire le temps d'exécution de l'algorithme $\text{Temps} = \text{Nombre opérations} \times C \text{ sec}$. On note $f(n_1, \dots, n_m)$ la fonction qui, pour $m \in \mathbb{N}$ tailles de paramètres, retourne la complexité en temps.

Nous allons en particulier nous concentrer sur la complexité **dans le pire cas**. Le principe est de décompter le maximum d'opérations pouvant advenir dans chaque situation : une boucle s'exécute du début à la fin et c'est toujours la branche conditionnelle qui nécessite le plus d'opérations qui s'exécute.

Exemple

Nous allons calculer la complexité en temps dans le pire cas pour cet algorithme simple, qui parcourt une liste de nombres et, si le nombre est pair, ajoute 1 à ce nombre.

Algorithm 0: Ajout de 1

Data: L liste, n taille de L

```
1 for  $i = 0$  to  $n - 1$  do
2    $c \leftarrow L[i]$ 
3   if  $c \% 2 == 0$  then
4      $L[i] \leftarrow c + 1$ 
5 return  $L$ 
```

Analysons l'algorithme 0 :

- ligne 1 : l'initialisation compte pour 1 opération, puis toutes les lignes dans le corps de la boucle vont être exécutées n fois :
 - ligne 2 : 1 opération (affectation)
 - ligne 3 : 2 opérations (modulo et comparaison)
 - ligne 4 : 2 opérations (addition et affectation), nous la comptons pour chaque tour de boucle pour respecter le pire cas
 - ligne 1 : la maintenance de la boucle compte pour 4 opérations ($i = i + 1$ et $i \leq n - 1$)
- ligne 5 : le retour compte pour 1 opération.

La complexité de l'algorithme 0 est donc $f(n) = 1 + (n - 1) \times (1 + 2 + 2 + 4) + 1$, ce qui donne $f(n) = 4n - 7$.

Notation "grand $\mathcal{O}$ "

Dans ce TP, nous allons travailler avec de grandes tailles de paramètres. Lorsque c'est le cas, les constantes additives et multiplicatives des complexités ont un impact moindre face aux tailles de paramètres : ce sont plutôt ces dernières qui vont définir le **comportement asymptotique** de la complexité. Nous utiliserons alors la notation $\mathcal{O}(\Gamma(f(n)))$, avec Γ renvoyant le membre dominant de $f(n)$ (voir les échelles de comparaison asymptotique¹), lorsque nous comparerons les complexités des différents algorithmes.

Pour reprendre l'analyse de complexité avec le "grand $\mathcal{O}$ " pour l'algorithme 0, nous avons :

- ligne 1 : $\mathcal{O}(1)$, puis $\mathcal{O}(n)$ fois :
 - ligne 2 : $\mathcal{O}(1)$
 - ligne 3 : $\mathcal{O}(1)$
 - ligne 4 : $\mathcal{O}(1)$
 - ligne 1 : $\mathcal{O}(1)$ } équivalent à $\mathcal{O}(1)$
- ligne 5 : $\mathcal{O}(1)$

Ce qui donne une complexité de $\mathcal{O}(1) + \mathcal{O}(n) \times \mathcal{O}(1) + \mathcal{O}(1)$, et donc de $\mathcal{O}(n)$.

1. https://fr.wikipedia.org/wiki/Comparaison_asymptotique

1 Algorithmes de tri (/28)

Dans cette partie, nous allons appréhender théoriquement l'utilité du parallélisme via deux algorithmes de tri : le tri par sélection et le tri fusion. Nous considérerons uniquement des listes de nombres entiers, sans doublons.

1.1 Tri par sélection (/16)

Le principe du tri par sélection est de parcourir une liste pour trouver son minimum, de le placer en tête de liste, puis de répéter le processus sur la partie de liste non triée. Voici le pseudo-code de cet algorithme :

Algorithm 1: Tri par sélection séquentiel

Data: L liste, n taille de L

```
1 for  $i = 0$  to  $n - 2$  do
     $min \leftarrow i$ 
    for  $j = i + 1$  to  $n - 1$  do
        if  $L[j] < L[min]$  then
5          $min \leftarrow j$ 
     $tmp \leftarrow L[min]$ 
     $L[min] \leftarrow L[i]$ 
     $L[i] \leftarrow tmp$ 
9 return  $L$ 
```

Question 1 (/1,5)

Pour chaque ligne, comptez le nombre d'opérations (affectations, comparaisons et arithmétiques) dans le pire cas.

Question 2 (/1,5)

Calculez le nombre d'opérations que l'algorithme 1 effectue dans le pire cas. Déduisez la complexité en temps de l'algorithme 1, avec la notation "grand $\mathcal{O}$ ".

Question 3 (/4)

On exécute maintenant cet algorithme en utilisant 3 threads, 1 thread principal A et 2 threads secondaires B et C. L'algorithme se comporte comme suit :

- Parallèlement, B et C cherchent respectivement les 1er et 2ème plus petits éléments de la liste, les sauvegardent dans des variables globales, envoient un signal à A, puis attendent un signal de A avant de recommencer la recherche sur la nouvelle liste.
- Lorsque A reçoit les 2 signaux, il procède séquentiellement à l'échange des 2 plus petits nombres au début de la liste, puis envoie un signal à B et C.

Pour les besoins de l'exercice, lorsque C rencontre le plus petit élément de la liste, la comparaison renvoie "False". Aussi, dans la première boucle, i avance avec un pas de 2.

Complétez l'exécution de cet algorithme sur la liste $L = [6, 3, 1, 2, 5, 4]$ de taille $n = 6$ sur le tableau d'exécution suivant, où 1 unité de temps permet de réaliser 1 opération.

(Vous joindrez au rapport une photo qualitative du tableau ci-dessous, rempli, pour cette question.)

Question 4 (/2)

Comptez le nombre d'opérations effectuées par chaque thread puis le nombre total d'opérations (opérations de $A + B + C$), et comparez-le au nombre total d'opérations nécessaires à l'algorithme séquentiel. Qu'en déduisez-vous ? Quelles lignes de l'algorithme le thread A aurait pu exécuter pour économiser plus d'opérations ?

Question 5 (/1)

Comparez cette fois-ci le temps d'exécution du tri de L avec le temps qu'il aurait fallu pour la trier entièrement séquentiellement. Donnez le rapport $\frac{\text{Temps Algo Séquentiel}}{\text{Temps Algo Parallèle}}$, appelé speed-up.

Question 6 (/1)

Que pourrions-nous modifier, dans l'algorithme parallèle, pour réduire encore plus le temps d'exécution ?

Question 7 (/2,5)

Donnez le temps qu'aurait pris l'algorithme si nous avions utilisé 3 threads secondaires, puis donnez le speed-up (commencez par compter le nombre de boucles sur i et sur j). Même question pour 4 threads secondaires.

Pensez-vous que le résultat aurait été le même si la liste avait été de taille 8 ? Qu'en déduisez-vous ?

Question 8 (/0,5)

Que se serait-il passé si nous n'avions utilisé qu'un seul thread secondaire ?

Question 9 (/1)

Donnez l'expression de la complexité en temps de l'algorithme pour une liste de taille n et pour m threads, avec n très grand et $m \leq n$. Comptez avant tout le nombre de boucles sur i et sur j .

Question 10 (/1)

La complexité de cet algorithme parallèle est en $\mathcal{O}\left(n + \frac{n^2}{m}\right)$. Quelles est la valeur de m qui optimise la complexité ? Trouvez-vous cela réaliste ?

1.2 Tri fusion (/12)

Le tri fusion est un exemple classique d'algorithme "diviser pour régner". Le principe est de découper la liste à trier en 2 sous-listes, trier les sous-listes, puis les fusionner en mettant les éléments dans le bon ordre.

On définit les 3 algorithmes suivants :

Algorithm 2: SousListes

Data: L liste, n taille L , $a < n$ taille de $L1$

```
1  $L1, L2 \leftarrow []$ 
  for  $i = 0$  to  $a - 1$  do
     $L1[i] \leftarrow L[i]$ 
  for  $i = a$  to  $n - 1$  do
5    $L2[i - a] \leftarrow L[i]$ 
6 return  $L1, L2$ 
```

Algorithm 3: Fusion

Data: $L1$ liste, $n1$ taille de $L1$, $L2$ liste, $n2$ taille de $L2$

```
1  $L \leftarrow []$ 
   $i, j \leftarrow 0$ 
   $L1[n1], L2[n2] \leftarrow \infty$ 
  while  $i \leq n1 - 1 \parallel j \leq n2 - 1$  do
5   if  $L1[i] < L2[j]$  then
     $L[i + j] \leftarrow L1[i]$ 
     $i \leftarrow i + 1$ 
  else
     $L[i + j] \leftarrow L2[j]$ 
10   $j \leftarrow j + 1$ 
11 return  $L$ 
```

Algorithm 4: Tri

Data: L liste, n taille de L

```
1 if  $n == 1$  then
  return  $L$ 
   $n1 \leftarrow \lfloor \frac{n}{2} \rfloor$ 
   $n2 \leftarrow \lceil \frac{n}{2} \rceil$ 
5  $L1, L2 \leftarrow \text{SousListes}(L, n, n1)$ 
   $L1\_Trie \leftarrow \text{Tri}(L1, n1)$ 
   $L2\_Trie \leftarrow \text{Tri}(L2, n2)$ 
   $L\_Trie \leftarrow \text{Fusion}(L1\_Trie, n1, L2\_Trie, n2)$ 
9 return  $L\_Trie$ 
```

Question 1 (/0,5)

Donnez la complexité en "grand $\mathcal{O}$ " de l'Algorithme 2.

Question 2 (/0,5)

Donnez la complexité en "grand $\mathcal{O}$ " de l'Algorithme 3 en fonction de n_1 et n_2 .

Question 3 (/1,5)

Donnez la complexité des lignes 5, 6, 7 et 8 de l'Algorithme 4. Déduisez l'expression du nombre d'opérations de cet algorithme, que l'on note $T(n)$, en fonction de $T(\frac{n}{2})$. (Attention : $T(n)$ n'est pas la même chose que la notation $\mathcal{O}(n)$)

Question 4 (/1)

À l'aide du *Master Theorem*, dont vous pouvez trouver l'énoncé ici ², calculez la complexité de l'Algorithme 4, en précisant la base du logarithme.

Question 5 (/1,5)

Comparez la complexité du tri fusion avec celles des tris par sélections séquentiel et parallèle. À partir de combien de threads secondaires m le tri sélection parallèle est-il plus rapide que le tri fusion ? Que cela donne-t-il pour une liste de taille 100 ?

Question 6 (/1)

Nous allons paralléliser le tri fusion. Premièrement, à chaque passage dans la fonction *Tri*, nous appelleront 2 threads secondaires pour réaliser en parallèle les lignes 6 et 7 de l'Algorithme 4, puis nous réaliserons la fusion de manière séquentielle. Donnez la nouvelle complexité (calculez $T(n)$ puis appliquez le *Master Theorem*).

Question 7 (/2)

Combien de threads seraient lancés au 1er passage dans *Tri* ? Au 2ème et 3ème passages ? Au n -ième ? Combien y a-t-il de niveaux d'appels à la fonction *Tri* ? En déduire le nombre de threads secondaires lancés en tout. Trouvez plusieurs astuces pour n'utiliser que n , puis $\frac{n}{2}$ threads. (Rappel : $\sum_{i=1}^n a^i = \frac{a(a^n-1)}{a-1}$)

1.2.1 Question 8 (/1)

Cette fois-ci, à la ligne 5 de *Tri*, nous allons diviser la liste en m sous-listes en séquentiel, puis nous allons lancer m threads en parallèle, puis fusionner les sous-listes en séquentiel. Exprimez puis calculez $T(n)$. Calculez aussi le nombre de threads nécessaires dans tout l'algorithme. Est-il plus rapide de diviser L en plus de 2 sous-listes ?

Question 9 (/2,5)

Notre algorithme n'est toujours pas convaincant : il requiert $m = n$ threads pour fonctionner. En fixant le nombre de threads m , expliquez ou écrivez en pseudo-code un algorithme de tri fusion parallèle qui n'utiliserait que ces m threads secondaires (ne modifier que la partie "tri"). Détaillez sa complexité.

2. https://fr.wikipedia.org/wiki/Master_theorem

Question 10 (/0,5)

Supposons que nous avons accès à un grand nombre de threads, quelles parties de *Tri* faut-il désormais changer pour améliorer la complexité ?